

Algorithmes et Pensée Computationnelle

Algorithmes de graphes

Le but de cette séance est de comprendre le fonctionnement des graphes et d'appliquer des algorithmes courants sur des graphes simples.

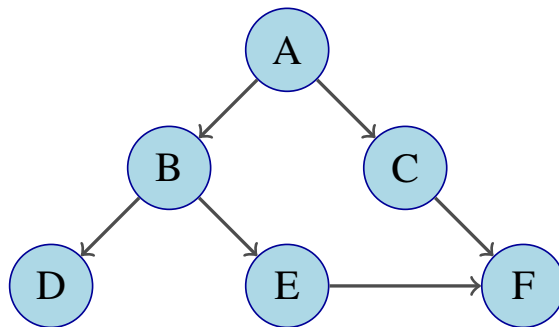
Le code présenté dans les énoncés se trouve sur Moodle, dans le dossier `code`. Le temps indiqué (🕒) est à titre indicatif.

1 Parcours en largeur — Breadth-First Search

Question 1: (🕒 10 minutes) Adjacency list et adjacency matrix : Python

L'adjacency list (ou liste de contiguïté) et l'adjacency matrix (ou matrice de contiguïté) sont les deux méthodes dont nous disposons pour représenter un graphe.

Utilisez ces deux méthodes de représentations pour stocker le graphe ci-dessous dans un programme Python :



Question 2: (🕒 20 minutes) Breadth-First Search algorithm : Python

Nous allons maintenant nous intéresser au premier algorithme portant sur les graphes : **Breadth-First Search**. Le but du Breadth-First Search est de trouver tous les sommets atteignables à partir d'un sommet de départ.

Implémentez l'algorithme en suivant les étapes suivantes :

1. Initialisez:
 - une `file` (queue FIFO — de type liste) qui contient initialement le sommet de départ.
 - une liste, appelée `visited`, de sommets visités initialement le sommet de départ.
 - une liste, appelée `order`, qui stocke l'ordre d'enlèvement de la `file` — sommets proches au sommet de départ au début de la liste.
2. Tant que la `file` n'est pas vide, répétez les spécifications suivantes:
 - (a) Retirez le premier sommet de la `file` (utilisez `pop`) et appelez-le `u`.
 - (b) Ajoutez `u` à `order`
 - (c) Pour chaque sommet adjacent `v` de `u`: si `v` n'est ni dans `visited`, alors :
 - Insérez `v` à la fin de la `file`.
 - Insérez `v` à la fin de `visited`.
3. Quand la `file` devient vide, tous les sommets atteignables depuis le sommet de départ ont été visités en largeur.
4. Retournez `order`.

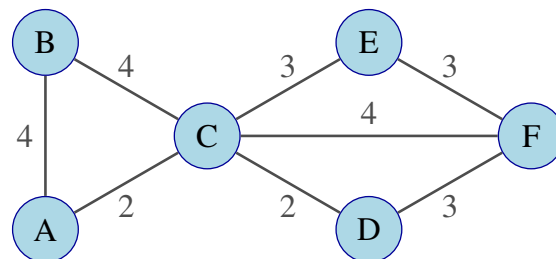
2 Arbre couvrant minimum (ACM) — Minimum Spanning tree (MST)

Nous allons maintenant nous intéresser à l'**algorithme de Kruskal**. Ce dernier s'applique uniquement aux **weighted graphs** (ou graphes pondérés). Ces derniers sont des graphes où les arêtes ont des poids, représentant par exemple une distance. L'algorithme de Kruskal a pour but de trouver un **arbre couvrant minimum** (ACM). Un graphe connexe est un graphe dans lequel il existe au moins un chemin entre n'importe quelle paire de sommets. Un arbre couvrant minimum S de G est un sous-graphe connexe de G tel que :

1. $V' = V$, c'est-à-dire que tous les sommets de G sont aussi dans S
2. (V', E') ne contient pas de cycle (pas de cycle dans S)
3. S est le graphe satisfaisant (1) et (2) et ayant la plus petite somme des poids
4. Pour chaque paire de sommets dans G , S contient le chemin *minimax* entre ces deux sommets. Le chemin *minimax* est un chemin pour lequel le poids de l'arête la plus lourde est minimisé.

Question 3: (🕒 5 minutes) Algorithme de Kruskal: Papier

Appliquez l'algorithme de Kruskal au graphe suivant:



Question 4: (🕒 30 minutes) Lecture du code — Algorithme de Kruskal: Python

Vous trouverez ci-dessous l'algorithme de Kruskal implémenté en Python (disponible sur Moodle). Lisez le code afin de vous assurer que vous avez bien compris le fonctionnement:

```
1 # Question 4
2
3 class Graph:
4     def __init__(self, vertices): # permet de créer un graphe lorsqu'on écrit
5         # p.ex. Graph(6), il faut notamment indiquer le nombre de sommets
6         self.V = vertices # le nombre de sommets
7         self.edges = [] # liste de tuples (u, v, w) ou w est le poids de
8         # l'arête entre u et v
9         self.parent = list(range(vertices)) # une liste [0, .., vertices - 1]
10        self.rank = [0]*vertices # une liste [0, ..,0] de longueur vertices
11
12    def add_edge(self, u, v, w): # ajoute une arête entre le sommet u et v
13        # avec un poids w
14        self.edges.append((u, v, w))
15
16    def find(self, i): # Correspond à la fonction Find-set(x) du cours
17        if self.parent[i] == i:
18            return i
19        self.parent[i] = self.find(self.parent[i])
20        return self.parent[i]
21
22    def union(self, x, y): # Correspond à la fonction Union(x,y) du cours
23        x_root = self.find(x)
24        y_root = self.find(y)
25
26        if x_root == y_root:
27            return False # x et y appartiennent déjà au même ensemble, on ne
28            # peut pas fusionner leurs ensembles
```

```

25
26     if self.rank[x_root] < self.rank[y_root]:
27         self.parent[x_root] = y_root
28     elif self.rank[x_root] > self.rank[y_root]:
29         self.parent[y_root] = x_root
30     else:
31         self.parent[y_root] = x_root
32         self.rank[x_root] += 1
33
34     return True # x et y n'appartiennent pas au même ensemble, on peut
    fusionner leurs ensembles
35
36 def kruskal(g : Graph):
37     result = []
38     edges = sorted(g.edges, key=lambda item: item[2]) # trie les arêtes
    par poids croissant
39     i = 0 # l'étape de l'itération
40     e = 0 # nombre d'arêtes pendant la construction de l'ACM
41
42     # Tant que le nombre d'arêtes est inférieur à V-1, notre sous-graphe
    n'atteint pas tous les sommets -> on continue
43     while e < g.V - 1:
44         u, v, w = g.edges[i] # self.graph contient les arêtes par ordre
    croissant de poids, on commence avec i = 0
45         i = i + 1 # à l'itération suivante on voudra avoir la 2ème arête
    la plus légère, donc on incrémente
46
47         if g.union(u, v): # Si u et v font déjà parti du Minimum Spanning
    Tree, i.e. u et v appartiennent au même ensemble
48             # Alors on ne veut pas ajouter cette arête au minimum
    spanning-tree
49             e = e + 1 # Si u et v sont d'ensemble différent, on a atteint
    un sommet de plus donc on incrémente
50             result.append([u, v, w]) # On ajoute la nouvelle arête au
    résultat
51
52     return result
53
54 if __name__ == '__main__':
55     g = Graph(6)
56     g.add_edge(0, 1, 4)
57     g.add_edge(0, 2, 4)
58     g.add_edge(1, 2, 2)
59     g.add_edge(3, 4, 3)
60     g.add_edge(2, 5, 2)
61     g.add_edge(4, 5, 3)
62
63     result = kruskal(g)
64
65     for u, v, weight in result:
66         print(f"{u} - {v}: {weight}") # afficher le résultat

```

Question 5: (🕒 10 minutes) Social Network Analysis : Papier

Les graphes peuvent être utilisés pour représenter une multitude de choses. L'une d'entre elles est la représentation de votre réseau d'amis. Imaginez que vous possédiez une liste de vos amis ainsi que des amis de vos amis (qui ne sont pas nécessairement vos amis). Cette liste peut-être représentée sous forme de graphe. Dans ce graphe:

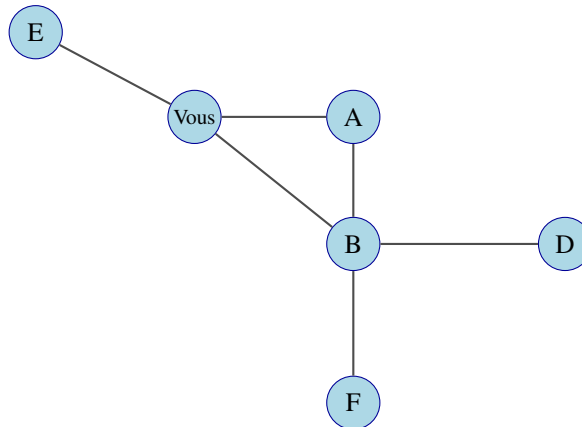
1. À quoi correspondent les arêtes et les sommets ?
2. Faut-il utiliser un graphe dirigé ?

Supposez que vous disposiez d'un graphe des relations sociales. Décrivez comment retrouver les éléments suivants :

1. Votre ami qui a le plus d'amis
2. Découvrir quels amis à vous se connaissent
3. Listez vos amis qui pourraient vous présenter quelqu'un que vous ne connaissez pas (ami d'ami qui n'est pas votre ami)

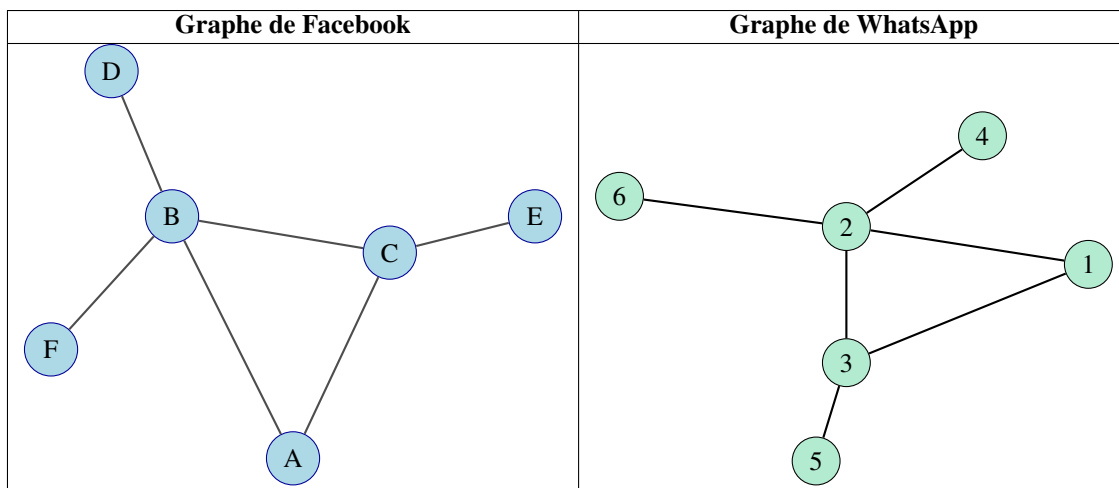
Vous voudriez désormais ajouter une nouvelle personne sur ce graphe, mais cette dernière n'est ni votre ami, ni l'ami d'un de vos amis. Quel sera son degré dans le graphe ?

Retrouvez les éléments (1) à (3) dans le graphe ci-dessous :



Question 6: (🕒 5 minutes) **Reconnaître des réseaux semblables**

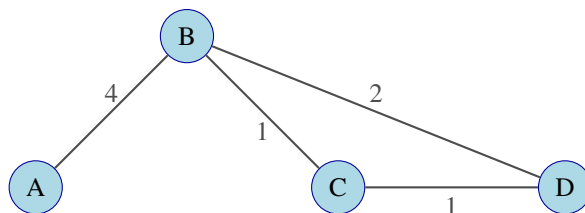
Supposons maintenant que vous travaillez chez Meta, qui a racheté Whatsapp, et que vous disposiez des deux graphes représentant respectivement Facebook et Whatsapp. Pouvez-vous déterminer visuellement à qui correspondent les individus de Facebook sur Whatsapp ?



Question 7: (🕒 5 minutes) Laquelle des affirmations suivantes est correcte ?

1. L'algorithme du parcours en largeur (breadth-first search) ne se termine pas s'il existe un cycle dans le graphe.
2. Pour $|G| \gg \|G\|$ (nombre de vertices beaucoup plus grand que nombre d'arêtes), la matrice de contiguïté (adjacency matrix) occupe moins d'espace que la liste de contiguïté (adjacency list).
3. Dans un graphe complet, le nombre d'arêtes est $\frac{n(n-1)}{2}$ et le degré moyen d'un sommet est n où $n = |G|$.
4. Dans l'algorithme de Kruskal, l'instruction UNION est utilisée afin de déterminer si deux sommets sont déjà liés dans l'arbre couvrant minimal et de les relier si ce n'est pas le cas.

Question 8: (🕒 10 minutes) Soit le graphe G donné ci-dessous, déterminer le nombre d'ensembles dis-joints (disjoint sets) après les premiers deux appels de UNION dans l'algorithme de Kruskal.



Algorithm 1 MST-KRUSKAL(G, w)

```

1:  $A \leftarrow \emptyset$ 
2: for each vertex  $v \in G.V$  do
3:   MAKE-SET( $v$ )
4: end for
5: sort the edges of  $G.E$  into nondecreasing order by weight  $w$ 
6: for each edge  $(u, v) \in G.E$ , taken in nondecreasing order by weight do
7:   if FIND-SET( $u$ )  $\neq$  FIND-SET( $v$ ) then
8:      $A \leftarrow A \cup \{(u, v)\}$ 
9:     UNION( $u, v$ )
10:  end if
11: end for
12: return  $A$ 

```
